



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 01

Team No.: 11

Project Title: Multithreaded Linux Application Using POSIX Threads and Mutexes

Team Members

Roll Number	Student Name	Signature
2520030349	D. Rohini Tanmayi	Rohini
2520030051	M. Pranitha	Pranitha
2520030326	S. Manvi Vaishnavi	Vaishnavi

1. Abstract

The Multithreaded Linux Application Using POSIX Threads and Mutexes is a Linux-based application designed to demonstrate how multiple threads can execute concurrently while safely accessing shared resources. The project uses POSIX Threads (pthreads) to create and manage multiple threads and mutexes to prevent race conditions when threads access common data. The application focuses on practical concepts of Linux process and thread management, synchronization, and concurrent programming. It demonstrates how proper synchronization improves the reliability and consistency of multithreaded applications.

2. Problem Statement

In a multithreaded application, multiple threads may access and modify the same shared resource at the same time. Without proper synchronization, this can result in race conditions, inconsistent data, data corruption, and unexpected program behavior.

Therefore, there is a need to develop a Linux application that demonstrates concurrent execution and provides safe access to shared resources using POSIX synchronization mechanisms.

3. Objectives

1. To understand the concept of multithreading in Linux._
2. To create and manage multiple threads using POSIX threads.
3. To demonstrate concurrent execution of multiple tasks.
4. To identify and prevent race conditions.
5. To implement mutex-based synchronization for shared resources.

4. Proposed Methodology

The project will be designed and implemented as a multithreaded Linux application using C and POSIX Threads (pthreads). The application will divide a larger task into multiple smaller tasks that can be executed concurrently by different threads.

First, the main program will initialize the required shared resources and mutexes. Multiple threads will then be created using `pthread_create()`, with each thread assigned a specific task. When a thread needs to access or modify shared data, it will acquire the mutex using `pthread_mutex_lock()`. After completing the critical operation, the thread will release the mutex using `pthread_mutex_unlock ( )`. This ensures that only one thread accesses the shared resource at a time and prevents race conditions.

After all threads complete their assigned tasks, the main program will use `pthread_join()` to wait for their termination. The final results will then be displayed and verified for consistency. The application may also be tested with and without mutex synchronization to demonstrate the effects of race conditions and the importance of thread synchronization.

The overall flow will be:

Initialize resources → Create threads → Execute tasks concurrently → Lock shared resource → Perform operation → Unlock resource → Join threads → Display and verify results.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Multithreading	Allows multiple threads to execute tasks concurrently within the same process.
Thread creation and termination	pthread_create() creates threads, while pthread_exit() and pthread_join() manage thread termination and completion.
Concurrent execution	Enables multiple threads to perform independent tasks simultaneously, improving responsiveness and resource utilization.
Thread synchronization	Coordinates access to shared resources and prevents conflicts between simultaneously executing threads.
Critical sections	Identifies the part of the program where shared data is accessed or modified and must be protected.
Thread scheduling	Determines how the Linux scheduler allocates CPU time among the threads.
Mutex / Mutual Exclusion	pthread_mutex_lock() and pthread_mutex_unlock() ensure that only one thread accesses the shared resource at a time.

6. Individual Contribution

(Clearly specify the responsibility of each team member in the project.)

Roll Number	Student Name	Individual Responsibility

7. Tools / Platforms / Software Used

(List the programming languages, operating system, libraries, tools, and software planned for the project.)

Tool / Platform / Software	Purpose
Linux / Ubuntu	
C / C++	

8. Expected Outcome

The project is expected to result in a functional multithreaded Linux application that creates and executes multiple threads concurrently using POSIX Threads. Each thread will perform an assigned task while accessing shared resources in a controlled manner.

The application will use mutex locks to protect critical sections and prevent race conditions, ensuring that shared data remains accurate and consistent. The output will display the creation, execution, and completion of individual threads, along with the final result of the shared operation.

The project will demonstrate:

- Successful creation and execution of multiple threads.
- Concurrent execution of independent tasks.
- Safe access to shared resources using mutexes.
- Prevention of race conditions and inconsistent results.
- Proper synchronization and termination of threads.
- A clear and consistent final output after all threads complete.

9. GitHub Repository

GitHub Repository Link: https://github.com/manviivaishnavi/BotsCreate_OSSP_

Repository Created: yes

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: _____

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____